

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

- i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right ii. Cost of each move = 1 iii. Cannot pass through obstacles iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved. c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- c) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 — Hospital Doctor Shift Scheduling (Backtracking Search)

Problem Restatement:

Assign doctors D1, D2, D3 to shifts Morning, Afternoon, Night (one doctor per shift, one shift per doctor) such that:

#	Constraint	Type
i	D1 cannot work Night shift	Unary constraint on D1
ii	D2 must work before D3 (Morning < Afternoon < Night)	Binary ordering constraint between D2 and D3
iii	Only one doctor per shift	All-different / mutual exclusion constraint
iv	D3 cannot work Morning shift	Unary constraint on D3
v	All doctors must be assigned exactly one shift	Completeness constraint (total assignment)

1.1 Constraint Analysis and Prioritisation

- Unary constraints (i and iv) are checked first for each variable because they prune the domain immediately without needing any other variable to be assigned — this is the cheapest and most effective pruning, so they are applied as soon as a value is proposed.
- The all-different constraint (iii) is enforced structurally by only offering shifts that are not already in the 'used_shifts' set, so it is never actually violated during search (no wasted branches).
- The relational constraint (ii) $D2 < D3$ can only be checked once both D2 and D3 have values, so it is evaluated last, after D3 is tentatively assigned.
- This ordering (unary $\rightarrow$ all-different $\rightarrow$ relational) minimises the depth of the search tree at which a dead end is discovered, which is the central efficiency idea of backtracking with constraint propagation.

1.2 Backtracking Search Procedure

Variable order chosen: D1, D2, D3 (fixed order). Value order tried for every variable: Morning, Afternoon, Night. At every assignment attempt, constraints (i), (iii), (iv) and — once both D2 and D3 are set — (ii) are checked immediately (this is 'backtracking with forward constraint checking'). If a check fails, that branch is pruned and the algorithm backtracks to try the next value for the current variable, or the previous variable if all values are exhausted.

1.3 Full Step-by-Step Search Trace (generated by running the program below)

BACKTRACKING SEARCH TRACE (Variable order: D1, D2, D3)

Step 1: Try D1 = Morning -> constraint check passed (OK); assignment so far = {'D1': 'Morning'}

Step 2: Try D2 = Morning -> REJECTED (shift already assigned; constraint iii violated)

Step 3: Try D2 = Afternoon -> constraint check passed (OK); assignment so far = {'D1': 'Morning', 'D2': 'Afternoon'}

Step 4: Try D3 = Morning -> REJECTED (shift already assigned; constraint iii violated)

Step 5: Try D3 = Afternoon -> REJECTED (shift already assigned; constraint iii violated)

Step 6: Try D3 = Night -> constraint check passed (OK); assignment so far = {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

Step 7: COMPLETE ASSIGNMENT -> {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}
==> VALID SOLUTION

Step 8: Try D2 = Night -> constraint check passed (OK); assignment so far = {'D1': 'Morning', 'D2': 'Night'}

Step 9: Try D3 = Morning -> REJECTED (shift already assigned; constraint iii violated)

Step 10: Try D3 = Afternoon -> REJECTED (D2 must be before D3
(Morning<Afternoon<Night))

Step 11: Try D3 = Night -> REJECTED (shift already assigned; constraint iii violated)

Step 12: Try D1 = Afternoon -> constraint check passed (OK); assignment so far = {'D1': 'Afternoon'}

Step 13: Try D2 = Morning -> constraint check passed (OK); assignment so far = {'D1': 'Afternoon', 'D2': 'Morning'}

Step 14: Try D3 = Morning -> REJECTED (shift already assigned; constraint iii violated)

Step 15: Try D3 = Afternoon -> REJECTED (shift already assigned; constraint iii violated)

Step 16: Try D3 = Night -> constraint check passed (OK); assignment so far = {'D1': 'Afternoon', 'D2': 'Morning', 'D3': 'Night'}

Step 17: COMPLETE ASSIGNMENT -> {'D1': 'Afternoon', 'D2': 'Morning', 'D3': 'Night'}
==> VALID SOLUTION

Step 18: Try D2 = Afternoon -> REJECTED (shift already assigned; constraint iii violated)

Step 19: Try D2 = Night -> constraint check passed (OK); assignment so far = {'D1': 'Afternoon', 'D2': 'Night'}

Step 20: Try D3 = Morning -> REJECTED (D3 cannot work Morning shift)

Step 21: Try D3 = Afternoon -> REJECTED (shift already assigned; constraint iii violated)

Step 22: Try D3 = Night -> REJECTED (shift already assigned; constraint iii violated)

Step 23: Try D1 = Night -> REJECTED (D1 cannot work Night shift)

=====

=====

Total valid solutions found: 2

Solution 1: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

Solution 2: {'D1': 'Afternoon', 'D2': 'Morning', 'D3': 'Night'}

First solution returned by the search (used as final schedule): {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

1.4 Final Valid Schedule

The search discovers two schedules that satisfy every constraint; the depth-first backtracking procedure returns the first one it finds:

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Verification against all constraints:

- (i) D1 = Morning $\neq$ Night ✓
- (ii) D2 = Afternoon (index 1) < D3 = Night (index 2) ✓
- (iii) Morning, Afternoon, Night are all distinct — one doctor per shift ✓
- (iv) D3 = Night $\neq$ Morning ✓
- (v) All three doctors assigned exactly one shift each ✓

(An alternate valid schedule also exists: D1 = Afternoon, D2 = Morning, D3 = Night — both are correct solutions to the CSP; the first one is reported as the final answer.)

1.5 Full Python Code — Backtracking Search

Q1: Backtracking Search - Doctor Shift Scheduling

SHIFTS = ["Morning", "Afternoon", "Night"]

ORDER = {"Morning": 0, "Afternoon": 1, "Night": 2}

```

step_counter = [0]
solutions = []

def constraints_ok(assignment):
    if assignment.get("D1") == "Night":
        return False, "D1 cannot work Night shift"
    if assignment.get("D3") == "Morning":
        return False, "D3 cannot work Morning shift"
    if "D2" in assignment and "D3" in assignment:
        if ORDER[assignment["D2"]] >= ORDER[assignment["D3"]]:
            return False, "D2 must be before D3 (Morning<Afternoon<Night)"
    return True, "OK"

def backtrack(assignment, remaining_vars, used_shifts):
    if not remaining_vars:
        step_counter[0] += 1
        print(f'Step {step_counter[0]}: COMPLETE ASSIGNMENT -> {assignment} ==>
VALID SOLUTION')
        solutions.append(dict(assignment))
        return

    var = remaining_vars[0]
    for shift in SHIFTS:
        step_counter[0] += 1
        if shift in used_shifts:
            print(f'Step {step_counter[0]}: Try {var} = {shift} -> REJECTED (shift already
assigned; constraint iii violated)')
            continue
        assignment[var] = shift
        ok, reason = constraints_ok(assignment)
        if ok:
            print(f'Step {step_counter[0]}: Try {var} = {shift} -> constraint check passed
({reason}); assignment so far = {assignment}')
            used_shifts.add(shift)
            backtrack(assignment, remaining_vars[1:], used_shifts)
            used_shifts.remove(shift)
        else:
            print(f'Step {step_counter[0]}: Try {var} = {shift} -> REJECTED ({reason}')
            del assignment[var]

print("="*90)
print("BACKTRACKING SEARCH TRACE (Variable order: D1, D2, D3)")

```

```
print("="*90)
backtrack({}, ["D1", "D2", "D3"], set())

print("\n" + "="*90)
print(f"Total valid solutions found: {len(solutions)}")
for i, s in enumerate(solutions, 1):
    print(f"  Solution {i}: {s}")
print("First solution returned by the search (used as final schedule):", solutions[0])
```

Question 2 — Robot Path Planning in a 5×5 Grid (Priority-Queue Search with Manhattan Heuristic)

Problem Restatement:

A robot must move from Start (S) to Goal (G) on a 5×5 grid, moving Up/Down/Left/Right at a cost of 1 per move, avoiding obstacles, and guided by the Manhattan-distance heuristic. Since the task requires both breadth-first-style level expansion (uniform step cost) AND a heuristic-ordered priority queue, this is implemented as a priority-queue-based Best-First Search (the informed generalisation of BFS, equivalent to A* when all step costs are equal), which guarantees the optimal path because the Manhattan-distance heuristic never overestimates the true remaining cost (it is admissible).

2.1 Grid Used for This Solution

A concrete 5×5 grid (rows 0–4, columns 0–4) is used to demonstrate the algorithm end-to-end:

S	.	.	.	.
.	#	.	#	.
.	.	.	#	.
.	#	.	#	.
.	.	.	.	G

- S = Start (0,0), G = Goal (4,4)
- Obstacles (#): (1,1), (1,3), (2,3), (3,1), (3,3)
- Move order explored at each node: Up, Down, Left, Right; each move cost = 1
- Heuristic $h(n) = |\text{row}(n) - \text{row}(G)| + |\text{col}(n) - \text{col}(G)|$ (Manhattan distance)
- Priority key $f(n) = g(n) + h(n)$, where $g(n)$ is the actual path cost from S to n

2.2 Step-by-Step Node Expansion and Priority-Queue Updates (generated by running the program below)

Grid layout (S=start, G=goal, #=obstacle, .=free):

S
. # . # .
. . . # .
. # . # .
. . . . G

=====

=====

STEP-BY-STEP NODE EXPANSION ($f = g + h$, h = Manhattan distance to Goal)

Step 1: POP node (0, 0) | g=0, h=8, f=8

Priority queue BEFORE this pop had 1 entries.

Expanding neighbors of (0, 0):

Up -> (-1, 0): out of bounds, skip

Down -> (1, 0): g=1, h=7, f=8 ==> pushed to priority queue

Left -> (0, -1): out of bounds, skip

Right -> (0, 1): g=1, h=7, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (2 entries), ordered by f:

node=(1, 0), g=1, h=7, f=8

node=(0, 1), g=1, h=7, f=8

Step 2: POP node (1, 0) | g=1, h=7, f=8

Priority queue BEFORE this pop had 2 entries.

Expanding neighbors of (1, 0):

Up -> (0, 0): already reached with g=0 <= 2, skip

Down -> (2, 0): g=2, h=6, f=8 ==> pushed to priority queue

Left -> (1, -1): out of bounds, skip

Right -> (1, 1): BLOCKED (obstacle), skip

Priority queue AFTER expansion (2 entries), ordered by f:

node=(2, 0), g=2, h=6, f=8

node=(0, 1), g=1, h=7, f=8

Step 3: POP node (2, 0) | g=2, h=6, f=8

Priority queue BEFORE this pop had 2 entries.

Expanding neighbors of (2, 0):

Up -> (1, 0): already reached with g=1 <= 3, skip

Down -> (3, 0): g=3, h=5, f=8 ==> pushed to priority queue

Left -> (2, -1): out of bounds, skip

Right -> (2, 1): g=3, h=5, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (3 entries), ordered by f:

node=(3, 0), g=3, h=5, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

Step 4: POP node (3, 0) | g=3, h=5, f=8

Priority queue BEFORE this pop had 3 entries.

Expanding neighbors of (3, 0):

Up -> (2, 0): already reached with g=2 <= 4, skip

Down -> (4, 0): g=4, h=4, f=8 ==> pushed to priority queue

Left -> (3, -1): out of bounds, skip

Right -> (3, 1): BLOCKED (obstacle), skip

Priority queue AFTER expansion (3 entries), ordered by f:

node=(4, 0), g=4, h=4, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

Step 5: POP node (4, 0) | g=4, h=4, f=8

Priority queue BEFORE this pop had 3 entries.

Expanding neighbors of (4, 0):

Up -> (3, 0): already reached with g=3 <= 5, skip

Down -> (5, 0): out of bounds, skip

Left -> (4, -1): out of bounds, skip

Right -> (4, 1): g=5, h=3, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (3 entries), ordered by f:

node=(4, 1), g=5, h=3, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

Step 6: POP node (4, 1) | g=5, h=3, f=8

Priority queue BEFORE this pop had 3 entries.

Expanding neighbors of (4, 1):

Up -> (3, 1): BLOCKED (obstacle), skip

Down -> (5, 1): out of bounds, skip

Left -> (4, 0): already reached with g=4 <= 6, skip

Right -> (4, 2): g=6, h=2, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (3 entries), ordered by f:

node=(4, 2), g=6, h=2, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

Step 7: POP node (4, 2) | g=6, h=2, f=8

Priority queue BEFORE this pop had 3 entries.

Expanding neighbors of (4, 2):

Up -> (3, 2): g=7, h=3, f=10 ==> pushed to priority queue

Down -> (5, 2): out of bounds, skip

Left -> (4, 1): already reached with g=5 <= 7, skip

Right -> (4, 3): g=7, h=1, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (4 entries), ordered by f:

node=(4, 3), g=7, h=1, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

node=(3, 2), g=7, h=3, f=10

Step 8: POP node (4, 3) | g=7, h=1, f=8

Priority queue BEFORE this pop had 4 entries.

Expanding neighbors of (4, 3):

Up -> (3, 3): BLOCKED (obstacle), skip

Down -> (5, 3): out of bounds, skip

Left -> (4, 2): already reached with g=6 <= 8, skip

Right -> (4, 4): g=8, h=0, f=8 ==> pushed to priority queue

Priority queue AFTER expansion (4 entries), ordered by f:

node=(4, 4), g=8, h=0, f=8

node=(2, 1), g=3, h=5, f=8

node=(0, 1), g=1, h=7, f=8

node=(3, 2), g=7, h=3, f=10

Step 9: POP node (4, 4) | g=8, h=0, f=8

Priority queue BEFORE this pop had 4 entries.

-> GOAL REACHED!

OPTIMAL PATH: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]

OPTIMAL PATH COST: 8 (number of moves = 8)

2.3 Optimal Path and Cost

Optimal Path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Optimal Path Cost = 8 (8 moves, each of unit cost). This matches the Manhattan distance between S and G in the obstacle-free case, confirming the path is optimal — the heuristic never led the search to overestimate, and no cheaper path around the obstacle wall exists.

2.4 Full Python Code — Priority-Queue (A*-style) Search

Q2: Robot Path Planning on 5x5 Grid

Priority-Queue-based (Best-First / A*-style) Search using Manhattan Distance heuristic

Moves: Up, Down, Left, Right ; cost of each move = 1

```
import heapq
```

```
GRID_SIZE = 5
```

```
S = (0, 0)      # Start (row, col)
```

```
G = (4, 4)      # Goal
```

```
OBSTACLES = {(1,1), (1,3), (2,3), (3,1), (3,3)}
```

```

def in_bounds(r, c):
    return 0 <= r < GRID_SIZE and 0 <= c < GRID_SIZE

def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

# Move order: Up, Down, Left, Right
MOVES = [("Up",(-1,0)), ("Down",(1,0)), ("Left",(0,-1)), ("Right",(0,1))]

def print_grid():
    print("Grid layout (S=start, G=goal, #=obstacle, .=free):")
    for r in range(GRID_SIZE):
        row = ""
        for c in range(GRID_SIZE):
            if (r,c) == S: row += "S "
            elif (r,c) == G: row += "G "
            elif (r,c) in OBSTACLES: row += "# "
            else: row += ". "
        print(row)
    print()

def search():
    counter = 0 # tie-breaker for heap
    g_costs = {S: 0}
    parent = {S: None}
    h0 = manhattan(S, G)
    frontier = [(h0 + 0, h0, 0, counter, S)] # (f, h, g, tie, node)
    heapq.heapify(frontier)
    visited = set()
    step = 0

    print("="*100)
    print("STEP-BY-STEP NODE EXPANSION (f = g + h, h = Manhattan distance to
Goal)")
    print("="*100)

    while frontier:
        step += 1
        f, h, g, _, node = heapq.heappop(frontier)

        print(f"\nStep {step}: POP node {node} | g={g}, h={h}, f={f}")
        print(f" Priority queue BEFORE this pop had {len(frontier)+1} entries.")

```

```

if node in visited:
    print(f" -> Already visited/expanded, skip.")
    continue
visited.add(node)

```

```

if node == G:
    print(f" -> GOAL REACHED!")
    path = []
    cur = node
    while cur is not None:
        path.append(cur)
        cur = parent[cur]
    path.reverse()
    print("\n" + "="*100)
    print(f"OPTIMAL PATH: {path}")
    print(f"OPTIMAL PATH COST: {g} (number of moves = {len(path)-1})")
    print("="*100)
    return path, g

```

```

print(f" Expanding neighbors of {node}:")

```

```

for name, (dr, dc) in MOVES:

```

```

    nr, nc = node[0]+dr, node[1]+dc

```

```

    neighbor = (nr, nc)

```

```

    if not in_bounds(nr, nc):

```

```

        print(f"    {name:5s} -> {neighbor}: out of bounds, skip")
        continue

```

```

    if neighbor in OBSTACLES:

```

```

        print(f"    {name:5s} -> {neighbor}: BLOCKED (obstacle), skip")
        continue

```

```

    new_g = g + 1

```

```

    if neighbor not in g_costs or new_g < g_costs[neighbor]:

```

```

        g_costs[neighbor] = new_g

```

```

        parent[neighbor] = node

```

```

        counter += 1

```

```

        nh = manhattan(neighbor, G)

```

```

        nf = new_g + nh

```

```

        heapq.heappush(frontier, (nf, nh, new_g, counter, neighbor))

```

```

        print(f"    {name:5s} -> {neighbor}: g={new_g}, h={nh}, f={nf} ==> pushed

```

```

to priority queue")

```

```

    else:

```

```

        print(f"    {name:5s} -> {neighbor}: already reached with
g={g_costs[neighbor]} <= {new_g}, skip")

    pq_snapshot = sorted(frontier)
    print(f"    Priority queue AFTER expansion ({len(pq_snapshot)} entries), ordered by
f:")
    for entry in pq_snapshot:
        ff, hh, gg, _, nn = entry
        print(f"        node={nn}, g={gg}, h={hh}, f={ff}")

    print("No path found.")
    return None, None

print_grid()
search()

```

Question 3 — Autonomous Rescue Robot in a Partially Observable Building

3(a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Effect on the Robot's Decision-Making
4-directional movement only	Restricts the branching factor of every search node to at most 4 successors, simplifying the action set but also limiting how quickly the robot can route around obstacles.
Blocked cells (obstacles)	These cells must be excluded from the successor function; since they are not known in advance, the robot can only prune them once sensed, which can force costly re-planning if a previously assumed-free path turns out to be blocked.
Limited sensing (adjacent cells only)	The robot cannot compute a full global optimal plan in advance (its knowledge is incomplete); it must interleave sensing and acting, i.e. plan-act-replan, characteristic of an online search agent in a partially observable environment.
Movement cost 1, risky-zone cost +2 (total 3)	Turns the problem into a weighted-graph shortest-path problem rather than plain step-counting; the robot must trade off a shorter route through a risky zone against a longer but cheaper safe route, which uniform-cost/A*-style evaluation handles naturally through $g(n)$.
Minimise total cost while ensuring safe navigation	Establishes the robot's performance measure (cost minimisation) jointly with a safety constraint (avoid unnecessary risk), i.e. a constrained optimisation objective rather than pure cost minimisation.
Dynamic knowledge update (online search)	The robot cannot rely on a static, precomputed model of the environment; every move must update its internal map, and its local decision procedure (e.g., LRTA*) must be able to revise earlier cost estimates when new information contradicts them.

3(b) Solution Approach: Online Search Agent (Local LRTA*-style Search)

Because the environment is only partially observable (sensing is limited to adjacent cells) and can only be discovered incrementally, a purely offline algorithm such as static A* cannot be used — the full state space is not known before the robot starts moving. The appropriate strategy is an Online Search Agent, implemented here as a Learning Real-

Time A* (LRTA*)-style local search combined with Uniform-Cost evaluation of already-known cells. The steps are:

- 1. Sense: at the current cell, observe the status (free / obstacle / risky) of all reachable adjacent cells and update the robot's internal knowledge map.
- 2. Evaluate: for every known, non-obstacle neighbour n , compute $f(n) = \text{step_cost}(n) + h(n)$, where step_cost is 1 for a normal cell and 3 for a risky cell (1 base + 2 penalty), and $h(n)$ is the Manhattan distance to the goal.
- 3. Select: choose the neighbour with the lowest $f(n)$; ties are broken in favour of the least-visited cell, to avoid the agent looping forever between two equally attractive cells (this is the standard LRTA* anti-oscillation rule).
- 4. Act: move to the chosen cell and accumulate its step cost into the running total cost.
- 5. Repeat steps 1–4 until the current cell equals the goal G (the survivor's location), continuously updating the knowledge map as previously unseen cells are sensed.

This approach is preferred over plain Uniform-Cost Search (UCS) because UCS assumes the entire graph/cost structure is known in advance; here the robot must act under uncertainty and revise its plan as obstacles and risky zones are discovered, which only an online search formulation supports.

Step-by-Step Simulation Trace (generated by running the program below)

ONLINE SEARCH AGENT TRACE (limited sensing, dynamic knowledge update)Step

1: Robot at (0, 0).

Sensing adjacent cells...

Sensed Down -> (1,0): free

Sensed Right -> (0,1): free

Decision: move Down to (1, 0) (step_cost=1, $f=\text{cost}+\text{heuristic}=8$)

Step 2: Robot at (1, 0). Sensing adjacent cells...

Sensed Up -> (0,0): free

Sensed Down -> (2,0): free

Sensed Right -> (1,1): obstacle

Decision: move Down to (2, 0) (step_cost=1, $f=\text{cost}+\text{heuristic}=7$)

Step 3: Robot at (2, 0). Sensing adjacent cells...

Sensed Up -> (1,0): free

Sensed Down -> (3,0): free

Sensed Right -> (2,1): risky

Decision: move Down to (3, 0) (step_cost=1, $f=\text{cost}+\text{heuristic}=6$)

Step 4: Robot at (3, 0). Sensing adjacent cells...

Sensed Up -> (2,0): free

Sensed Down -> (4,0): free

Sensed Right -> (3,1): obstacle

Decision: move Down to (4, 0) (step_cost=1, f=cost+heuristic=5)

Step 5: Robot at (4, 0). Sensing adjacent cells...

Sensed Up -> (3,0): free

Sensed Right -> (4,1): free

Decision: move Right to (4, 1) (step_cost=1, f=cost+heuristic=4)

Step 6: Robot at (4, 1). Sensing adjacent cells...

Sensed Up -> (3,1): obstacle

Sensed Left -> (4,0): free

Sensed Right -> (4,2): free

Decision: move Right to (4, 2) (step_cost=1, f=cost+heuristic=3)

Step 7: Robot at (4, 2). Sensing adjacent cells...

Sensed Up -> (3,2): risky

Sensed Left -> (4,1): free

Sensed Right -> (4,3): free

Decision: move Right to (4, 3) (step_cost=1, f=cost+heuristic=2)

Step 8: Robot at (4, 3). Sensing adjacent cells...

Sensed Up -> (3,3): free

Sensed Left -> (4,2): free

Sensed Right -> (4,4): free

Decision: move Right to (4, 4) (step_cost=1, f=cost+heuristic=1)

SURVIVOR REACHED at (4, 4)

PATH TAKEN: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]

TOTAL COST INCURRED: 8

3(c) Application of AI Concepts

Intelligent Agent (PEAS description)

PEAS Element	Description for This Rescue Robot
Performance measure	Minimise total movement/risk cost while safely locating the survivor; reward reaching G, penalise unnecessary risky-zone traversal and wasted moves.
Environment	5×5 building grid with static obstacles, risky zones, a fixed goal (survivor) — but unknown to the robot until sensed.
Actuators	Movement actuators that move the robot Up, Down, Left, or Right by one cell.
Sensors	Local proximity sensors that report the status (free / obstacle / risky) of the four adjacent cells only.

Rationality

The agent is rational because, given the percepts available to it at each step (its limited sensor readings) and its built-in performance measure, it always selects the action that maximises its expected performance — i.e., the neighbour that minimises cost-so-far plus estimated remaining distance. It is not omniscient (it does not know the whole grid in advance), so it can occasionally make a locally reasonable choice that turns out sub-optimal only when new information invalidates it — which is precisely why it replans continuously, remaining rational under uncertainty rather than being irrational or clairvoyant.

Environment Type

Property	Classification	Justification
Observability	Partially observable	Only adjacent cells are sensed; the full grid state is never directly available.
Determinism	Deterministic	Each move has a fully predictable outcome (no randomness in movement).
Episodic vs Sequential	Sequential	Each move affects future options and cost, so current decisions have long-term consequences.
Static vs Dynamic	Static	Obstacles and risky zones do not move or change while the robot is deciding (the environment itself is fixed; only the robot's knowledge of it changes).
Discrete vs Continuous	Discrete	Grid cells, discrete actions (4 directions), and discrete time steps.

Property	Classification	Justification
Single vs Multi-agent	Single-agent	Only the rescue robot acts in the environment (no adversary or cooperating agent modelled).

3(d) Justification of the Chosen Approach

- Optimality under uncertainty: LRTA*-style online search is proven to eventually find a path to the goal in a safely explorable, finite state space, updating its cost estimates as it learns — this matches the requirement to 'update its knowledge dynamically'.
- Correct cost handling: by encoding the risky-zone penalty directly into step_cost, the same $f = g + h$ evaluation used for normal Uniform-Cost/A* reasoning is preserved, so the robot naturally prefers a longer safe route over a shorter risky one only when the extra distance costs less than the risk penalty — this directly satisfies 'minimise total cost while ensuring safe navigation'.
- Feasibility with limited sensors: unlike static A*/BFS, which require the full map upfront, the online agent only ever needs local percepts, matching the robot's actual sensing capability described in the problem.
- Real-time responsiveness: because decisions are made one step at a time from local information, the robot can react immediately to newly discovered obstacles without having to discard and recompute an entire global plan, which is essential in a disaster-response scenario where every second matters.
- Therefore, an Online Search Agent using an LRTA*-style local best-first evaluation is the most suitable strategy for this constrained, partially observable, cost-sensitive rescue scenario.

Full Python Code — Online Search Agent Simulation

```
# Q3: Online Search Agent for Disaster-Affected Building (Partially Observable Environment)
# The agent (rescue robot) only senses the 4 adjacent cells at each step.
# It performs an ONLINE version of Uniform-Cost / A*-like local search (similar to LRTA*):
# at every cell it looks only at what it currently knows, picks the most promising
# unblocked neighbor by (path-cost-so-far + heuristic), moves there, and updates its
# knowledge base as new cells become visible. Risky-zone cells cost 2 to enter instead of 1.
```

```
GRID_SIZE = 5
```

```
S = (0, 0)
```

```
G = (4, 4)
```

OBSTACLES = {(1,1), (2,2), (3,1)} # unknown to the robot until sensed
 RISKY = {(2,1), (3,2), (1,3)} # extra cost +2 when entered (total cost 3), also
 unknown until sensed

```
def in_bounds(r, c):
    return 0 <= r < GRID_SIZE and 0 <= c < GRID_SIZE
```

```
def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])
```

```
def move_cost(cell):
    return 3 if cell in RISKY else 1 # base cost 1, +2 penalty if risky
```

```
MOVES = [("Up",(-1,0)), ("Down",(1,0)), ("Left",(0,-1)), ("Right",(0,1))]
```

```
def sense(cell, known_map):
    """Robot can only observe cells adjacent to its current cell (limited sensing)."""
    r, c = cell
    for name, (dr, dc) in MOVES:
        nr, nc = r+dr, c+dc
        if in_bounds(nr, nc):
            status = "obstacle" if (nr,nc) in OBSTACLES else ("risky" if (nr,nc) in RISKY
else "free")
            known_map[(nr, nc)] = status
```

```
def online_agent():
    known_map = {S: "free"}
    current = S
    total_cost = 0
    path = [current]
    visited_count = {}
    step = 0

    print("="*100)
    print("ONLINE SEARCH AGENT TRACE (limited sensing, dynamic knowledge
update)")
    print("="*100)
```

```
while current != G:
    step += 1
    sense(current, known_map)
    print(f"\nStep {step}: Robot at {current}. Sensing adjacent cells...")
```

```

for name, (dr, dc) in MOVES:
    nr, nc = current[0]+dr, current[1]+dc
    if (nr, nc) in known_map:
        print(f"  Sensed {name:5s} -> ({nr},{nc}): {known_map[(nr,nc)]}")

    # choose best known, reachable, unblocked neighbor greedily (LRTA*-style local
decision)
    candidates = []
    for name, (dr, dc) in MOVES:
        nr, nc = current[0]+dr, current[1]+dc
        nxt = (nr, nc)
        if nxt in known_map and known_map[nxt] != "obstacle":
            step_cost = move_cost(nxt) if known_map[nxt]=="risky" else 1
            f = step_cost + manhattan(nxt, G)
            visits = visited_count.get(nxt, 0)
            candidates.append((f, visits, name, nxt, step_cost))

    candidates.sort(key=lambda x: (x[0], x[1]))
    chosen = candidates[0]
    f, visits, name, nxt, step_cost = chosen
    print(f"  Decision: move {name} to {nxt} (step_cost={step_cost},
f=cost+heuristic={f})")

    total_cost += step_cost
    current = nxt
    path.append(current)
    visited_count[current] = visited_count.get(current, 0) + 1

    if step > 40:
        print("Stopping: step limit reached (safety cutoff for this trace).")
        break

print("\n" + "="*100)
print(f'SURVIVOR REACHED at {current}')
print(f'PATH TAKEN: {path}')
print(f'TOTAL COST INCURRED: {total_cost}')
print("="*100)

online_agent()

```